1 Positive depth and nonempty finite axes

Source: `PrimeTensor/Depth.lean`

What this file establishes

The file fixes the indexing vocabulary that the rest of the development is built on: a type of *depths* (dimensions and ranks), a family of coordinate *axes* indexed by a depth, a fold over an axis, and the type of index tuples of a given rank.

Its single organising decision is that *nothing starts at zero*. Depth has no zeroth constructor; consequently every axis carries at least one coordinate, every tensor has at least one dimension, and every index tuple has at least one component. This is what lets fold be defined without an identity element for its binary operation, which in turn is what makes the later multiplicative layer — where the carrier is the strictly positive reals and there is no additive unit to fall back on — expressible at all.

1.1 Depth

Definition 1.1 (Depth, `PrimeTensor.Depth`)**.** Depth is the inductive type generated by the two constructors

$$\mathsf{one} : \mathrm{Depth}, \qquad \mathsf{succ} : \mathrm{Depth} \to \mathrm{Depth}.$$

```
inductive Depth where
  | one : Depth
  | succ : Depth → Depth
  deriving DecidableEq, Repr
```

This is the Peano presentation of the natural numbers with the base case moved from 0 to 1. Writing

$$|\mathsf{one}| \equiv 1, \qquad |\mathsf{succ}\, d| \equiv |d| + 1,$$

the map $d \mapsto |d|$ is a bijection $\mathrm{Depth} \to \mathbb{Z}_{\geq 1}$; we use it silently below whenever a depth is read as a number. Because Depth is a first-order inductive type with no parameters, Lean derives decidable equality and a printing instance for it mechanically, and structural induction on Depth is available: to prove $P(d)$ for all d it suffices to prove $P(\mathsf{one})$ and $P(d) \Rightarrow P(\mathsf{succ}\, d)$.

Definition 1.2 (Small depths, `Depth.two`, `Depth.three`)**.**

$$\mathsf{two} \equiv \mathsf{succ}\, \mathsf{one}, \qquad \mathsf{three} \equiv \mathsf{succ}\, \mathsf{two}.$$

These are abbreviations, not new data: $|\mathsf{two}| = 2$ and $|\mathsf{three}| = 3$, and each unfolds definitionally to a stack of constructors. three is the depth at which the Navier–Stokes layer of the project operates.

Design note 1.3. The absence of a zero constructor is a positive design choice and not an oversight. Had Depth been $\mathbb{N}$, then $\mathrm{Axis}\, 0$ (Definition 1.4) would be empty, fold (Definition 1.8) would need an identity element to return in that case, and every downstream lemma about a fold would carry a side condition excluding the degenerate dimension. Ruling the degenerate case out at the level of the index type discharges all of those obligations at once, by making the bad case unrepresentable rather than merely excluded.

1.2 Axes

Definition 1.4 (Axis, `PrimeTensor.Axis`)**.** Axis is the inductive family Depth → Type generated by

$$\mathsf{first} : \mathrm{Axis}\, d \quad (\text{for every } d : \mathrm{Depth}), \qquad \mathsf{next} : \mathrm{Axis}\, d \to \mathrm{Axis}\,(\mathsf{succ}\, d).$$

```
inductive Axis : Depth → Type where
  | first {d : Depth} : Axis d
  | next  {d : Depth} : Axis d → Axis (.succ d)
```

Note the asymmetry between the two constructors. first is available at every depth, so it is the coordinate that always exists; next is available only at a depth that is literally a successor, so it can only push an existing coordinate of Axis d into the strictly larger Axis (succ d).

Proposition 1.5 (Axes are inhabited)**.** *For every* $d : \mathrm{Depth}$ *the type* Axis d *is nonempty.*

Proof. The term $\mathsf{first} : \mathrm{Axis}\, d$ is well formed for every d, since the depth argument of first is unconstrained. □

Proposition 1.6 (Cardinality of an axis)**.** *For every* $d : \mathrm{Depth}$ *the type* Axis d *is finite, with exactly* $|d|$ *elements.*

Proof. By structural induction on d (Definition 1.1).

Base case $d = \mathsf{one}$. Every element of an inductive family is a constructor application. An element of Axis one built by next would have type Axis (succ d') for some d', so its depth index would have to satisfy $\mathsf{one} = \mathsf{succ}\, d'$; distinct constructors of Depth produce distinct values, so no such d' exists. Hence first is the only element, and $\#\mathrm{Axis}\,\mathsf{one} = 1 = |\mathsf{one}|$.

Inductive step $d = \mathsf{succ}\, d'$. The two constructors are jointly surjective onto Axis (succ d') and have disjoint images, and next is injective, both again because constructors of an inductive type are disjoint and injective. Therefore

$$\mathrm{Axis}\,(\mathsf{succ}\, d') \;\cong\; \{\mathsf{first}\} \sqcup \mathrm{Axis}\, d',$$

so by the induction hypothesis $\#\mathrm{Axis}\,(\mathsf{succ}\, d') = 1 + |d'| = |\mathsf{succ}\, d'|$. □

Notation 1.7. Proposition 1.6 lets us name the elements of Axis d in order. For $n = |d|$ put

$$i_1 \equiv \mathsf{first}, \qquad i_{k+1} \equiv \mathsf{next}\, i_k \quad (1 \le k < n),$$

so that Axis $d = \{i_1, \ldots, i_n\}$ with all i_k distinct. Equivalently $i_k = \mathsf{next}^{k-1}(\mathsf{first})$, and this term is well typed in Axis d exactly when $k \le n$: each application of next consumes one succ from the depth index, so d must be at least a $(k-1)$-fold successor. The enumeration is thus cut off by the depth itself, with no side condition to carry.

1.3 Folding over an axis

Definition 1.8 (Fold, `PrimeTensor.Axis.fold`)**.** Let A be a type and $* : A \to A \to A$ a binary operation. For $d : \mathrm{Depth}$ and $f : \mathrm{Axis}\, d \to A$ define $\mathrm{fold}_*(d, f) : A$ by structural recursion on d:

$$\mathrm{fold}_*(\mathsf{one}, f) = f(\mathsf{first}), \tag{1}$$

$$\mathrm{fold}_*(\mathsf{succ}\, d, f) = f(\mathsf{first}) * \mathrm{fold}_*\big(d,\ f \circ \mathsf{next}\big). \tag{2}$$

```
def fold {A : Type} (op : A → A → A) :
    (d : Depth) → (Axis d → A) → A
  | .one,    f => f .first
  | .succ d, f => op (f .first) (fold op d (fun i => f (.next i)))
```

The recursion is structural in d and therefore terminates: the recursive call is made at the immediate predecessor of $\mathsf{succ}\, d$. The function argument is re-indexed rather than shrunk — $f \circ \mathsf{next}$ has domain $\mathrm{Axis}\, d$ because $\mathsf{next} : \mathrm{Axis}\, d \to \mathrm{Axis}\,(\mathsf{succ}\, d)$ — which is exactly why the recursion can be driven by the depth alone.

Design note 1.9. fold takes no seed value and $*$ is required to be neither associative, nor commutative, nor unital. A fold over a possibly-empty index type must return something on the empty index type, and the only uniform choice is an identity element for $*$; requiring one would force every carrier used with fold to be a monoid. Proposition 1.5 removes the empty case, so fold is total on a bare magma. This is what makes fold usable over the strictly positive reals under multiplication, and over min and max, none of which is a monoid in the form the later files need.

Lemma 1.10 (Base equation, `PrimeTensor.Axis.fold_one`)**.** *For all $*$ and all $f : \mathrm{Axis}\,\mathsf{one} \to A$,*

$$\mathrm{fold}_*(\mathsf{one}, f) = f(\mathsf{first}).$$

Proof. Both sides are the same term after unfolding fold once: (1) is the defining equation of fold at the constructor one, and the match on d reduces because one is a constructor application in head normal form. The two sides are therefore definitionally equal, and the proof is rfl. □

Lemma 1.11 (Successor equation, `PrimeTensor.Axis.fold_succ`)**.** *For all $*$, all d : Depth, and all $f : \mathrm{Axis}\,(\mathsf{succ}\, d) \to A$,*

$$\mathrm{fold}_*(\mathsf{succ}\, d, f) = f(\mathsf{first}) * \mathrm{fold}_*\big(d,\ f \circ \mathsf{next}\big).$$

Proof. As in Lemma 1.10: this is (2), the defining equation at the constructor succ, and the match reduces without any hypothesis on d. Definitional equality again, proved by rfl. □

Remark 1.12. Both equations hold by reflexivity, so as *propositions* they are content-free. They are stated, and tagged `@[simp]`, for a mechanical reason: Lean's simplifier rewrites with declared lemmas, not by unfolding recursive definitions on its own. Registering the two equations makes simp evaluate any fold whose depth is a concrete numeral, which is what downstream files rely on. Together they also constitute the complete case analysis on d, so no third equation is possible.

Proposition 1.13 (What a fold computes)**.** *Let $n = |d|$ and let $i_1, \ldots, i_n$ enumerate $\mathrm{Axis}\, d$ as in Notation 1.7. Then*

$$\mathrm{fold}_*(d, f) = f(i_1) * \big(f(i_2) * (\cdots * f(i_n))\big),$$

the right-nested $*$-expression over the coordinates in order, with no identity element at the innermost position.

Proof. By induction on d. For $d = \mathsf{one}$ we have $n = 1$ and Lemma 1.10 gives $f(i_1)$. For $d = \mathsf{succ}\, d'$, Lemma 1.11 gives $f(\mathsf{first}) * \mathrm{fold}_*(d', f \circ \mathsf{next})$; the enumeration of $\mathrm{Axis}\,(\mathsf{succ}\, d')$ is first followed by the next-image of the enumeration of $\mathrm{Axis}\, d'$, so the induction hypothesis applied to $f \circ \mathsf{next}$ rewrites the second factor as the right-nested expression over $f(i_2), \ldots, f(i_n)$. □

Remark 1.14. Proposition 1.13 is the informal reading of Definition 1.8; it is stated here for the reader and is not among the declarations of `PrimeTensor/Depth.lean`. The bracketing it exhibits is fixed, not incidental: without associativity of $*$ the value of a fold depends on it.

1.4 Index tuples

Definition 1.15 (Index tuple, `PrimeTensor.IndexTuple`)**.** For a dimension $dim : \mathrm{Depth}$ define $\mathrm{IndexTuple}(dim, -)$, a map $\mathrm{Depth} \to \mathrm{Type}$, by structural recursion on the rank:

$$\begin{aligned} \mathrm{IndexTuple}(dim, \mathsf{one}) &= \mathrm{Axis}\ dim, \\ \mathrm{IndexTuple}(dim, \mathsf{succ}\, r) &= \mathrm{Axis}\ dim \times \mathrm{IndexTuple}(dim, r). \end{aligned}$$

```
def IndexTuple (dim : Depth) : Depth → Type
  | .one   => Axis dim
  | .succ r => Axis dim × IndexTuple dim r
```

Unlike Axis, this is not an inductive family but a function into Type defined by recursion, so its two equations hold definitionally and a tuple is literally a nested pair — no constructor or projection layer sits between $\mathrm{IndexTuple}(dim, \mathsf{three})$ and $\mathrm{Axis}\ dim \times (\mathrm{Axis}\ dim \times \mathrm{Axis}\ dim)$. Both arguments are depths, but they play different roles: dim is the size of each coordinate axis and is held fixed, while the second argument is the rank and drives the recursion. A rank-one index tuple is a single axis index rather than a one-element product, so there is no empty tuple and no unit type in the encoding.

Proposition 1.16 (Cardinality of the index set)**.** $\mathrm{IndexTuple}(dim, r)$ *is finite with* $|dim|^{|r|}$ *elements; in particular it is nonempty.*

Proof. By induction on r. For $r = \mathsf{one}$, Proposition 1.6 gives $\#\mathrm{Axis}\ dim = |dim| = |dim|^1$. For $r = \mathsf{succ}\, r'$, the cardinality of a product is the product of the cardinalities, so

$$\#\mathrm{IndexTuple}(dim, \mathsf{succ}\, r') = |dim| \cdot |dim|^{|r'|} = |dim|^{|r'|+1} = |dim|^{|\mathsf{succ}\, r'|}.$$

Nonemptiness follows since $|dim| \geq 1$, or directly from Proposition 1.5 by pairing. □

Corollary 1.17 (The intended case)**.** $\mathrm{IndexTuple}(\mathsf{three}, \mathsf{three})$ *has* $3^3 = 27$ *elements: the index set of a rank-three tensor in three dimensions.*

Summary of the correspondence

Lean declaration	Kind	Here
`PrimeTensor.Depth`	inductive type	Def. 1.1
`PrimeTensor.Depth.two`	definition	Def. 1.2
`PrimeTensor.Depth.three`	definition	Def. 1.2
`PrimeTensor.Axis`	inductive family	Def. 1.4
`PrimeTensor.Axis.fold`	definition	Def. 1.8
`PrimeTensor.Axis.fold_one`	theorem	Lem. 1.10
`PrimeTensor.Axis.fold_succ`	theorem	Lem. 1.11
`PrimeTensor.IndexTuple`	definition	Def. 1.15

Every declaration of `PrimeTensor/Depth.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition; the two theorems it does state are proved by `rfl`. The mathematical content of the file is therefore entirely in its definitions, and the results proved about them here — Propositions 1.5, 1.6, 1.13 and 1.16 — are consequences of those definitions stated for the reader rather than formalised obligations.